

Tailscale + Pi-hole en Portainer

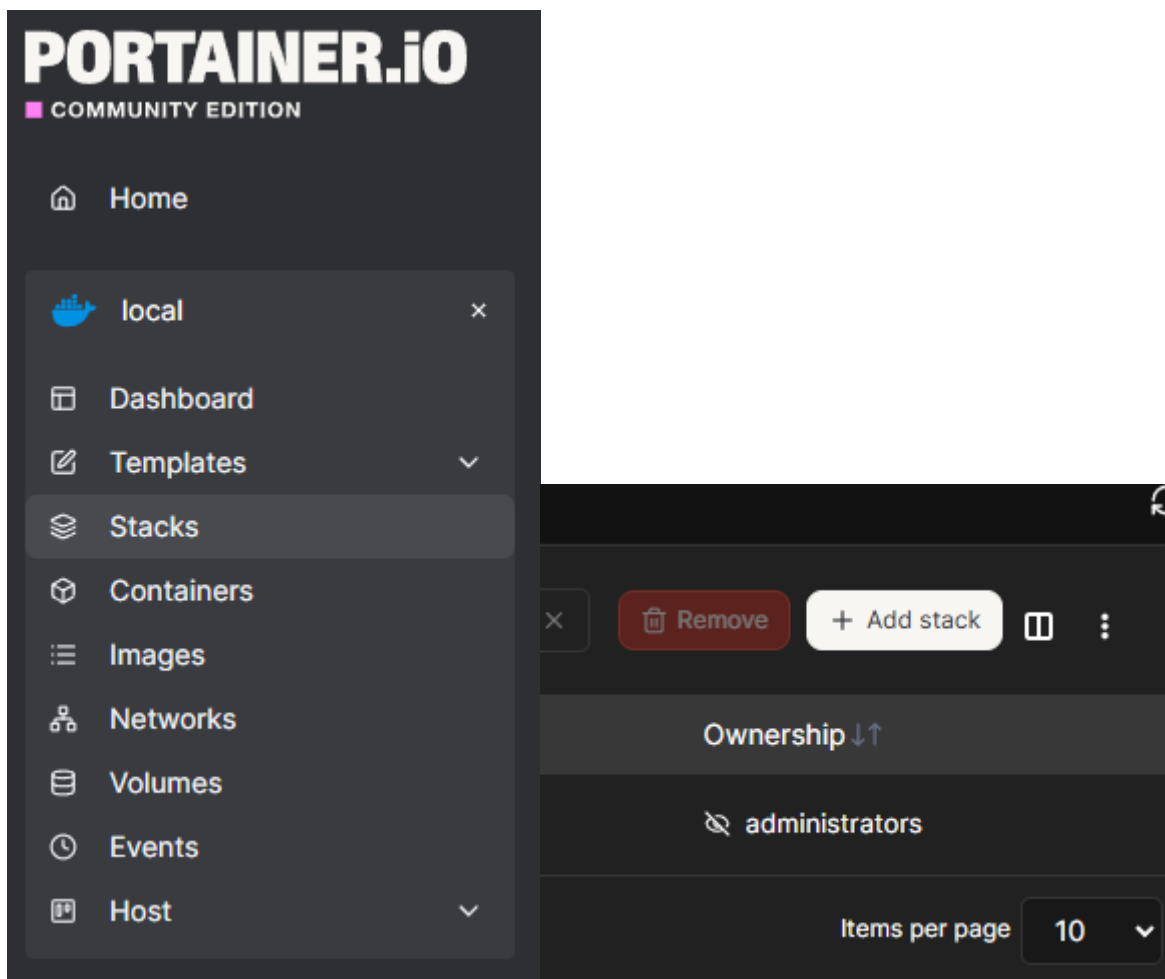
1. Requisitos Previos y Preparación del Sistema

- **Sistema base:** Raspberry Pi con Docker y Portainer instalados.
- **Activación del enrutamiento (Sysctl):** Para permitir que la Raspberry Pi actúe como router de red (necesario para el *Exit Node*), se configuró el núcleo de Linux ejecutando en la terminal (SSH):

```
echo 'net.ipv4.ip_forward = 1' | sudo tee -a /etc/sysctl.d/99-tailscale.conf
echo 'net.ipv6.conf.all.forwarding = 1' | sudo tee -a /etc/sysctl.d/99-
tailscale.conf
sudo sysctl -p /etc/sysctl.d/99-tailscale.conf
```

2. Despliegue en Portainer

- Crearemos un nuevo **Stack** en Portainer usando la arquitectura *Sidecar*. Tailscale funciona con la red del host (`network_mode: host`) para gestionar las rutas físicas, y Pi-hole se engancha directamente al contenedor de Tailscale (`network_mode: "service:tailscale"`). Para ello se crea nuevo stack desde portainer con un docker compose.
- Para crear el nuevo stack, desde el panel de Portainer le damos a stacks y a la derecha le damos a + Add stack



A continuación le damos un nombre y ponemos en el editor el docker compose:

```
version: "3.7"

services:
  tailscale:
    image: tailscale/tailscale:latest
    container_name: tailscale
    restart: unless-stopped
    network_mode: host
    environment:
      - TS_HOSTNAME=raspberrypi
      - TS_AUTHKEY=${TS_AUTHKEY}
      - TS_STATE_DIR=/var/lib/tailscale
      - TS_ROUTES=192.168.1.0/24
      - TS_EXTRA_ARGS=--advertise-exit-node
      - TS_USERSPACE=false
      - TS_AUTH_ONCE=true
    volumes:
      - tailscale_data:/var/lib/tailscale
    devices:
      - /dev/net/tun:/dev/net/tun
    cap_add:
      - NET_ADMIN
      - NET_RAW
```

```

pids_limit: 100
healthcheck:
  test: ["CMD", "tailscale", "status"]
  interval: 30s
  timeout: 5s
  retries: 3

pihole:
  image: pihole/pihole:latest
  container_name: pihole
  restart: unless-stopped
  network_mode: "service:tailscale"
  depends_on:
    tailscale:
      condition: service_healthy
  environment:
    - TZ=Europe/Madrid
    - FTLCONF_webserver_api_password=${PIHOLE_PASSWORD}
    - FTLCONF_dns_listeningMode=ALL
    - PIHOLE_UID=1000
    - PIHOLE_GID=1000
    - FTLCONF_dns_specialDomains_iCloudPrivateRelay=true
  volumes:
    - pihole_data:/etc/pihole
  cap_add:
    - SYS_NICE
  pids_limit: 100
  healthcheck:
    test: ["CMD", "dig", "@127.0.0.1", "pi.hole"]
    interval: 30s
    timeout: 5s
    retries: 3

volumes:
  tailscale_data:
  pihole_data:

```

Como vemos en el compose, tenemos dos variables: `TS_AUTHKEY=${TS_AUTHKEY}` y `FTLCONF_webserver_api_password=${PIHOLE_PASSWORD}`. Esto lo debemos añadir abajo del editor en el apartado Environment variables antes del darle a "Deploy the stack", para que así nuestras contraseñas sean privadas y no se vean, poniendo nuestras credenciales.

Para el caso de la contraseña de tailscale, se hace de la siguiente forma:

1. Nos dirigimos a la página oficial login.tailscale.com e iniciamos sesión
2. En el menú de la izquierda, hacemos clic en **Settings** y luego buscamos la pestaña o sección llamada **Keys**

3. Hacemos clic en el botón para **Generate auth key**
4. Marcamos las casillas que nos aparecen. Es muy recomendable marcar la opción de **Reusable** por si algún día necesitamos recrear el contenedor sin generar una clave nueva.

Para el caso de la contraseña de pi-hole, ponemos la contraseña con la que nos registramos en pi-hole la primera vez.

Environment variables

You may use [environment variables in your compose file](#). The environment variable values set below will be used as substitutions in the compose file. If your compose file contains the environment variables and their values (e.g. TAG=v1.5).

🕒 **stack.env file operation**

When deploying via **Repository**, the `stack.env` file must already reside in the Git repo.
When deploying via **Web editor**, **Upload** or **Custom template deployment**, the `stack.env` file is auto created from what you set below.

🔗 **Advanced mode**
🕒 Switch to advanced mode to copy & paste multiple variables

name *	value
TS_AUTHKEY	e.g. bar
PIHOLE_PASSWORD	e.g. bar

+ Add an environment variable ⬆️ Load variables from .env file

Como vemos también, para los que usan Iphone añadimos la siguiente línea para que no nos choque con el relay privado de Apple:

FTLCONF_dns_specialDomains_iCloudPrivateRelay=true

3. Configuración en el Panel Web de Tailscale

1. Aprobación de Rutas y Nodo de Salida:

- Entramos en nuestro navegador web, accedemos a la plataforma de administración en login.tailscale.com e iniciamos sesión con nuestras credenciales.
- Nos dirigimos a la sección **Machines** en el menú principal para ver la lista de dispositivos conectados de nuestra red privada virtual.
- Buscamos nuestra Raspberry Pi, hacemos clic en los tres puntos suspensivos (`...`) situados al final de su fila y seleccionamos **Edit route settings** (Editar ajustes de ruta).
- En la ventana emergente que aparece, activamos la casilla de **Subnet routes** para habilitar el rango de nuestra red doméstica (`192.168.X.X/24`), lo que nos permite acceder a cualquier equipo de casa de forma remota.
- Activamos también la casilla del **Exit node** (*Use as exit node*) para permitir que todo nuestro tráfico de internet se enrute y salga a través de nuestra Raspberry Pi cuando nos encontremos fuera de casa. Por último, guardamos los cambios haciendo clic en **Save**.

2. Asignación del Servidor DNS (Pi-hole):

- En el panel de control lateral izquierdo de Tailscale, hacemos clic en la sección **DNS**.

- Nos desplazamos hasta el apartado de **Nameservers**, hacemos clic en el botón **Add nameserver** y seleccionamos la opción **Custom**.
- Introducimos la dirección IP de Tailscale correspondiente a nuestra Raspberry Pi (por ejemplo, `100.x.x.x`) para que actúe como nuestro servidor DNS centralizado.
- Activamos la opción **Override local DNS** (Forzar DNS local) para obligar a todos nuestros dispositivos conectados a la VPN a utilizar nuestro Pi-hole de manera exclusiva, garantizando así el bloqueo automático de publicidad y rastreadores en cualquier lugar.

4. Sincronización y Ajustes de Ejecución

- Con el propósito de asegurar que el contenedor se aplicara de forma correcta y en caliente las directivas de red y el nodo de salida, ejecutamos el siguiente comando de inicialización forzada en el entorno de ejecución:

```
docker exec -it tailscale tailscale up --accept-routes --advertise-exit-node  
--force-reauth --netfilter-mode=on --accept-dns=false --advertise-  
routes=192.168.X.X/24 --hostname=raspberrypi
```